

Este último tramo del curso introduce varios temas importantes para completar la formación básica en desarrollo web con .NET.

Material de Estudio: Validación, Estado y Temas Adicionales en ASP.NET Core MVC

Profesor: Jorge Izaguirre **Materia:** Programacion .NET **Curso:** 6to Año - TÉCNICO EN PROGRAMACIÓN

Introducción

Ya hemos aprendido a construir el núcleo de una aplicación web MVC: hemos modelado nuestros datos, creado controladores para manejar la lógica, interactuado con bases de datos usando Entity Framework Core, y diseñado vistas con Razor y Tag Helpers. Ahora, vamos a explorar algunos temas adicionales que son cruciales para construir aplicaciones más robustas, seguras, eficientes y mantenibles.

En este módulo, reforzaremos las técnicas de validación, aprenderemos a manejar mensajes de estado entre diferentes peticiones, introduciremos los conceptos fundamentales de seguridad web (Autenticación y Autorización) y veremos cómo ASP.NET Core Identity nos ayuda. También cubriremos cómo registrar eventos importantes en nuestra aplicación (Logging), cómo optimizar la entrega de nuestros archivos CSS y JavaScript (Bundling y Minification), daremos una mirada conceptual a cómo desplegar nuestra aplicación en servidores reales, y finalmente, presentaremos los principios SOLID, guías fundamentales para escribir código orientado a objetos de alta calidad.

1. Refuerzo de Validación: Construyendo Formularios Robustos

Hemos visto cómo usar DataAnnotations y `ModelState` para validar la entrada del usuario. Es vital recalcar por qué este proceso es tan importante y cómo funcionan juntas las validaciones del cliente y del servidor.

- **¿Por qué validar SIEMPRE en el Servidor?**
 - **Seguridad:** La validación del lado del cliente (JavaScript en el navegador) puede ser **fácilmente evitada** por un usuario malintencionado (desactivando JS, modificando el HTML, enviando peticiones HTTP directamente). Si no validas en el servidor, datos incorrectos o maliciosos podrían entrar en tu sistema, corromper tu base de datos o causar fallos de seguridad.
 - **Integridad de Datos:** Asegura que solo datos que cumplen las reglas de negocio lleguen a tu base de datos, manteniendo la consistencia.
 - **Fuente de Verdad:** Las reglas definidas en tus modelos/ViewModels con DataAnnotations son la fuente única de verdad para la validación, usadas tanto por el servidor como (a través de los Tag Helpers) por el cliente.
- **El Flujo Completo:**
 1. **Definición:** Reglas en el ViewModel con DataAnnotations (`[Required]`, `[StringLength]`, etc.).
 2. **Cliente:** Tag Helpers (`input asp-for`, `span asp-validation-for`) generan atributos `data-val-*`. Scripts (`_ValidationScriptsPartial`) leen estos atributos y realizan validación en el navegador *antes* de enviar, dando feedback rápido.
 3. **Servidor:** En la acción `[HttpPost]`, el Model Binding intenta poblar el ViewModel. Luego, `ModelState.IsValid` verifica si hubo errores de binding Y si se cumplieron todas las DataAnnotations. **Esta es la validación definitiva.**
 4. **Respuesta:** Si `ModelState.IsValid` es *false*, se retorna la misma `View(viewModel)` para mostrar los errores (usando los `span asp-validation-for`). Si es *true*, se procesan los datos y se redirige (Patrón PRG).

2. Manejo de Estado Básico entre Peticiones: TempData

Recordemos que HTTP es un protocolo *sin estado*. Cada petición es independiente. Cuando usamos el patrón Post-Redirect-Get (PRG) para evitar reenvíos de formularios, ¿cómo podemos mostrar un mensaje de éxito o error al usuario *después* de la redirección? La información que ponemos en ViewBag o ViewData se pierde en la redirección porque son parte de una única petición/respuesta.

- **TempData al Rescate:**

- ¿Qué es? Es un mecanismo proporcionado por ASP.NET Core para almacenar datos que **persisten durante una redirección**, específicamente hasta la **siguiente petición HTTP**. Es ideal para "mensajes flash" (notificaciones temporales).
- ¿Cómo funciona? TempData utiliza un "proveedor" para almacenar los datos temporalmente entre peticiones. Por defecto, suele usar **Cookies encriptadas**, lo que significa que la información viaja ida y vuelta entre el servidor y el navegador. (Se puede configurar para usar el estado de Sesión del servidor si está habilitado).
- *Característica Clave:* Una vez que un valor de TempData es **leído** (accedido usando `@TempData["MiClave"]` en una vista o `TempData["MiClave"]` en un controlador), se marca automáticamente para ser **eliminado** al final de esa petición. ¡Solo dura una lectura!

- **Uso Práctico:**

1. **Guardar en la Acción POST (antes de redirigir):**

```
[HttpPost]
public async Task<IActionResult> Editar(int id, ProductoViewModel vm)
{
    if (id != vm.Id) return BadRequest();

    if (ModelState.IsValid)
    {
        // ... lógica para buscar y actualizar el producto ...
        try
        {
            await _context.SaveChangesAsync();
            TempData["MensajeExito"] = "¡Producto actualizado correctamente!"; // Guardar mensaje
            return RedirectToAction(nameof(Index)); // Redirigir (PRG)
        }
        catch (DbUpdateException ex)
        {
            _logger.LogError(ex, "Error al actualizar producto {ProductoId}", id);
            TempData["MensajeError"] = "Error al guardar los cambios en la base de datos.";
        }
    }

    // Si ModelState no es válido o hubo error al guardar, volver a la vista
    return View(vm);
}
```

2. Leer y Mostrar en la Vista GET (la que se muestra después de la redirección, ej:

`Index.cshtml` o en `_Layout.cshtml`):

```
@if (TempData.ContainsKey("MensajeExito")) // Mejor verificar si existe
antes de leer
{
    <div class="alert alert-success">
        @TempData["MensajeExito"] <!-- Al leerlo aquí, se marcará para
eliminar --%>
    </div>
}
@if (TempData.ContainsKey("MensajeError"))
{
    <div class="alert alert-danger">
        @TempData.Peek("MensajeError") <!-- Peek lo lee pero NO lo
marca para eliminar --%>
    </div>
    @{ TempData.Keep("MensajeError"); } <!-- Otra forma de conservarlo
una petición más si fuera necesario --%>
}
```

- **Limitaciones:** Diseñado para datos pequeños y temporales (como strings de mensajes). No usar para estado complejo de la aplicación. Si se basa en cookies, hay límites de tamaño y consideraciones de seguridad (aunque ASP.NET Core las encripta y firma).
- **Alternativa - Estado de Sesión (Session) (Conceptual):**
 - Permite almacenar datos en el **servidor** asociados a la **sesión** de un usuario específico (mientras navega por el sitio).
 - Requiere configuración explícita en `Program.cs` (`builder.Services.AddSession();` y `app.UseSession();`).
 - Útil para datos que deben persistir entre múltiples peticiones del mismo usuario (ej: un carrito de compras temporal).
 - Consume más recursos del servidor y puede complicar la escalabilidad si no se configura con almacenamiento distribuido (ej: Redis, SQL Server) en entornos con múltiples servidores web. Usar con moderación.

3. Fundamentos de Seguridad Web: Autenticación y Autorización

Proteger nuestra aplicación es fundamental. Los dos conceptos clave son:

- **Autenticación (AuthN - "¿Quién Eres?")**
 - El proceso de **verificar la identidad** de un usuario. Es el "login".
 - El usuario prueba quién es (generalmente con usuario/contraseña). El sistema lo verifica contra un almacén de usuarios (base de datos).
 - Si la verificación es exitosa, el sistema considera al usuario **autenticado** y normalmente establece una prueba de ello (como una cookie encriptada) para que el usuario no tenga que volver a

ingresar sus credenciales en cada petición.

- **Autorización (AuthZ - "¿Qué Puedes Hacer?")**

- Ocurre **después** de la autenticación.
- Es el proceso de **determinar si el usuario autenticado tiene permiso** para acceder a un recurso específico (una página, una API) o para realizar una determinada acción (crear, editar, borrar).
- Se basa en la identidad y en reglas definidas:
 - *Basada en Roles*: ¿El usuario tiene el rol "Administrador"? ¿O "Editor"?
 - *Basada en Políticas/Claims*: ¿El usuario tiene el "claim" (afirmación) de ser "MayorDe18"? ¿Pertenece al departamento "Ventas"?

- **ASP.NET Core Identity: Tu Sistema de Membresía Integrado**

- Es un framework completo dentro de ASP.NET Core que nos **simplifica enormemente** la implementación de AuthN y AuthZ. ¡Nos ahorra meses de trabajo y nos ayuda a hacerlo de forma segura!
- **¿Qué nos da?**
 - Manejo seguro de usuarios y contraseñas (almacenamiento hasheado).
 - Funcionalidad de Registro, Login, Logout.
 - Gestión de perfiles, roles, claims.
 - Recuperación de contraseña, confirmación de email.
 - Soporte para 2FA, logins externos (Google, etc.).
 - UI pre-construida (Páginas Razor) que podemos usar y personalizar.
- **Integración (Scaffolding):**
 1. **Prerrequisito:** Tener EF Core configurado (Identity guarda sus datos en tablas de BD).
 2. **Proceso:** En VS, **Agregar > Nuevo elemento con scaffold... > Identidad**. Se eligen las páginas a generar, se selecciona nuestro **DbContext**.
 3. **Resultado:** Se instalan paquetes NuGet, se modifica **Program.cs** (**AddDefaultIdentity**, **AddEntityFrameworkStores**, **UseAuthentication**, **UseAuthorization**), se crean las páginas en **Areas/Identity/Pages**.
 4. **Migración: ¡Esencial!** Ejecutar **Add-Migration NombreMigracionIdentity** y **Update-Database** para crear las tablas **AspNetUsers**, **AspNetRoles**, etc. en la BD.
- **Uso Básico:** La aplicación ahora tendrá páginas funcionales de Registro y Login. Para proteger controladores o acciones, usamos el atributo **[Authorize]**:

```
[Authorize] // Requiere que el usuario esté logueado para acceder a
CUALQUIER acción de este controlador
public class MiAreaSeguraController : Controller
{
    // ... acciones ...
}

// O proteger acciones individuales
public class HomeController : Controller
{
    [Authorize(Roles = "Admin,SuperUsuario")] // Requiere login Y tener
    alguno de estos roles
```

```
public IActionResult AdminPanel() { /* ... */ }
}
```

- Identity es muy extenso, esto es solo la puerta de entrada. Nos da una base de seguridad robusta sobre la cual construir.

4. Logging Básico: Registrando lo Importante

Cuando la aplicación corre en un servidor, no podemos depurar como en desarrollo. Necesitamos **registros (logs)** para entender qué sucede, diagnosticar problemas y monitorear la salud de la aplicación.

- **ILogger<T>**: La interfaz estándar de .NET para logging. Se obtiene por **Inyección de Dependencias**.

```
private readonly ILogger<MiServicio> _logger;
public MiServicio(ILogger<MiServicio> logger) { _logger = logger; }
```

- **Niveles de Severidad**: Para clasificar los mensajes.
 - *Trace* (más detallado) -> *Debug* -> *Information* -> *Warning* -> *Error* -> *Critical* (más severo).
- **Métodos**: `_logger.LogInformation("...");`, `_logger.LogError(ex, "Mensaje con excepción");`, etc.
- **Structured Logging**: Usar placeholders `{}` para los datos variables. Es la mejor práctica.

```
_logger.LogInformation("Procesando pedido ID {PedidoId} para cliente {ClienteId}", pedido.Id, pedido.ClienteId);
```

- **Configuración**: En `appsettings.json` se define el **nivel mínimo** a registrar para diferentes categorías y en `Program.cs` se configuran los **proveedores** (a dónde escribir: Console, Debug, File, Azure App Insights, etc.).

5. Bundling y Minification: Optimizando la Entrega de CSS/JS

Para que nuestras páginas web carguen más rápido:

- **Bundling (Empaquetado)**: Combina múltiples archivos `.css` en uno solo y múltiples `.js` en uno solo. **Reduce el número de peticiones HTTP.**
- **Minification (Minificación)**: Elimina caracteres innecesarios (espacios, comentarios) y acorta nombres en archivos `.css` y `.js`. **Reduce el tamaño de los archivos.**
- **En ASP.NET Core**: Se suelen usar herramientas en tiempo de compilación (ej: `BuildBundlerMinifier` NuGet package) o herramientas del ecosistema JS (Webpack, Vite) para generar archivos `.min.css` y `.min.js`. El `_Layout.cshtml` a menudo usa los **Tag Helpers de Entorno** (`<environment include="Development">` / `<environment exclude="Development">`) para enlazar los archivos individuales en desarrollo y los archivos optimizados (`.min`) en producción. El objetivo es **mejorar la performance**.

6. Despliegue (Deployment): Del Desarrollo a Producción

Desplegar es el proceso de llevar nuestra aplicación funcional desde nuestra máquina a un servidor accesible públicamente.

- **Publicar (`dotnet publish`):** Es el primer paso. Este comando compila la aplicación y empaqueta todos los archivos necesarios (DLLs, dependencias, `wwwroot`, configuración) en una carpeta (`publish`) lista para ser copiada al servidor.
- **Opciones de Hosting:**
 - **IIS (en Windows Server):** Servidor web de Microsoft. Actúa como reverse proxy para nuestra app ASP.NET Core (que corre con Kestrel). Requiere configurar el servidor IIS.
 - **Azure App Service (Nube PaaS):** Servicio gestionado en Azure. Nos abstraemos del servidor y SO. Fácil de escalar y desplegar (directo desde VS o con CI/CD desde Git/GitHub Actions). **Muy recomendado por su facilidad y potencia.**
 - **Otros:** Linux (con Nginx/Apache), Docker (contenedores).
- **Configuración por Entorno:** Usar `appsettings.Development.json`, `appsettings.Production.json` y **Variables de Entorno** (o Azure Key Vault) para manejar configuraciones distintas (como connection strings) y **nunca** guardar secretos en el repositorio Git.

7. Principios SOLID: Guía para un Buen Diseño Orientado a Objetos

Son 5 principios que nos ayudan a escribir código más mantenible, flexible y comprensible:

1. **S - Responsabilidad Única (SRP):** Una clase = Una razón para cambiar.
2. **O - Abierto/Cerrado (OCP):** Abierto a extensión, cerrado a modificación (usar abstracciones).
3. **L - Sustitución de Liskov (LSP):** Las clases derivadas deben poder sustituir a las bases sin romper nada.
4. **I - Segregación de Interfaces (ISP):** Interfaces pequeñas y específicas, no interfaces "gordas".
5. **D - Inversión de Dependencias (DIP):** Dependier de abstracciones (interfaces), no de implementaciones concretas (usar DI).

Seguirlos (con criterio) lleva a un software de mayor calidad a largo plazo.

Conclusión

Hemos cubierto temas esenciales que van más allá del flujo básico de MVC y EF Core. La validación robusta, el manejo adecuado del estado, la implementación de seguridad (AuthN/AuthZ), el registro de eventos, la optimización del front-end y la comprensión de cómo desplegar nuestra aplicación, junto con la aplicación de buenos principios de diseño como SOLID, son habilidades que distinguen a un desarrollador profesional. El camino del aprendizaje continúa, pero estas herramientas y conceptos te proporcionan una base sólida para construir aplicaciones .NET completas y de calidad.

Síntesis de una oración: Este material profundiza en la validación, el manejo de estado temporal con TempData, introduce la seguridad con ASP.NET Core Identity, el logging con ILogger, optimizaciones front-end, conceptos de despliegue en IIS/Azure y los principios de diseño SOLID para crear aplicaciones .NET robustas y mantenibles.

Síntesis de 5 puntos importantes:

1. La **Validación** es crucial y debe implementarse tanto en cliente (UX) como, obligatoriamente, en servidor (`ModelState.IsValid`) usando DataAnnotations.

2. **TempData** permite pasar mensajes flash entre peticiones después de redirecciones (Patrón PRG), solucionando la naturaleza stateless de HTTP para casos de uso específicos.
3. **Autenticación** (verificar quién es) y **Autorización** (verificar qué puede hacer) son fundamentales; **ASP.NET Core Identity** simplifica enormemente su implementación.
4. El **Logging** (**ILogger**, niveles, structured logging) es indispensable para diagnosticar y monitorear aplicaciones en producción.
5. Comprender el **Despliegue** (IIS, Azure), la optimización front-end (**Bundling/Minification**) y los principios **SOLID** son pasos clave hacia el desarrollo profesional.

41254